

Mini-projet 13 — Les context managers (with)

Pourquoi ce mini-projet ?

Tu utilises `with open(...)` depuis le MP01 sans savoir **ce qu'il fait vraiment**. Un `with` garantit qu'une ressource est **ouverte puis fermée — même si une erreur survient au milieu**. Au Module réseau, une socket qu'on oublie de fermer, c'est une fuite. Aujourd'hui tu comprends le mécanisme, et tu fabriques ton **propre** context manager (une `Connexion` qui s'ouvre et se ferme toute seule).

1. Le problème : fermer, même en cas d'erreur

Sans `with`, pour être sûr de fermer une ressource quoi qu'il arrive, il faut un `try/finally` (revu au MP05) :

```
f = open("data.txt")
try:
    traiter(f)          # si ÇA plante...
finally:
    f.close()           # ...le close() a QUAND MÊME lieu
```

C'est lourd et facile à oublier. Le `with` fait **exactement ça**, en une ligne :

```
with open("data.txt") as f:
    traiter(f)          # peu importe ce qui se passe ici...
# ...le fichier est fermé automatiquement en sortant du bloc, erreur ou pas
```

La garantie du `with` : « j'ouvre en entrant, je ferme en sortant, **même si ça explose au milieu** ». C'est le `finally` intégré, sans que tu aies à l'écrire.

2. Comment ça marche : `__enter__` et `__exit__`

N'importe quel objet devient utilisable dans un `with` s'il a **deux méthodes spéciales** :

Méthode	Quand elle s'exécute	Rôle
<code>__enter__(self)</code>	en entrant dans le <code>with</code>	préparer la ressource, et renvoyer ce que <code>as x</code> va recevoir
<code>__exit__(self, ...)</code>	en sortant (normalement OU sur erreur)	nettoyer / fermer

```
class Connexion:
    def __init__(self, cible):
        self.cible = cible

    def __enter__(self):
        print(f"connexion à {self.cible}")
        return self          # ce que "as c" recevra

    def __exit__(self, exc_type, exc_val, exc_tb):
        print(f"fermeture de {self.cible}")
        # les 3 paramètres décrivent l'erreur éventuelle (None si tout va bien)
```

```
with Connexion("10.0.0.5") as c:
    print(f"travail sur {c.cible}")
# Affiche :
# connexion à 10.0.0.5      <- __enter__
# travail sur 10.0.0.5     <- le corps du with
# fermeture de 10.0.0.5    <- __exit__
```

Le `as c` : il reçoit ce que `__enter__` a renvoyé. Ici `return self` → `c` est l'objet `Connexion` lui-même. (Pour `open`, `__enter__` renvoie le fichier → d'où `as f` = le fichier.)

3. La garantie en action : même sur erreur

```
with Connexion("10.0.0.5") as c:
    print("avant l'erreur")
    raise ValueError("boum")      # une erreur au milieu
    print("après (jamais atteint)")
# Affiche :
# connexion à 10.0.0.5
# avant l'erreur
# fermeture de 10.0.0.5      <- __exit__ s'exécute QUAND MÊME, avant que l'erreur remonte
```

C'est tout l'intérêt : la fermeture a lieu **même quand le bloc plante**. Une socket, un fichier, un verrou... tu es sûr qu'ils sont libérés. Aucune fuite.

4. La version rapide : @contextmanager

Écrire une classe pour ça, c'est parfois trop. Le module `contextlib` propose un raccourci avec un générateur (tu combines le MP10 et le MP11 !) :

```
from contextlib import contextmanager

@contextmanager
def connexion(cible):
    print(f"connexion à {cible}")    # tout ce qui est AVANT yield = __enter__
    yield cible                      # la valeur donnée à "as x"
    print(f"fermeture de {cible}")   # tout ce qui est APRÈS yield = __exit__

with connexion("10.0.0.5") as cible:
    print(f"travail sur {cible}")
```

Le **yield** coupe la fonction en deux : avant = préparation (entrée), après = nettoyage (sortie). C'est la même idée que la classe, en plus court.

🎯 Ta mission

Objectif : fabriquer ton propre context manager, en **classe**, qui simule une connexion (on ne fait pas encore de vrai réseau — ça, c'est le Module D).

- Écris `class Connexion` avec :
 - `__init__(self, cible)` ;
 - `__enter__(self)` qui affiche "connexion à <cible>" et renvoie `self` ;
 - `__exit__(self, exc_type, exc_val, exc_tb)` qui affiche "fermeture de <cible>" .
- Utilise-la dans un `with` normal, puis dans un `with` qui **lève une erreur** au milieu, pour vérifier que la fermeture a **quand même** lieu.

Tests à faire passer :

```
with Connexion("10.0.0.5") as c:
    print(f"scan de {c.cible}")
# -> connexion à 10.0.0.5
# scan de 10.0.0.5
# fermeture de 10.0.0.5

# Le test qui prouve la garantie :
try:
    with Connexion("10.0.0.6") as c:
        print("avant l'erreur")
        raise ValueError("boum")
except ValueError:
    print("erreur attrapée dehors")
# -> connexion à 10.0.0.6
# avant l'erreur
# fermeture de 10.0.0.6      (le __exit__ a bien tourné malgré l'erreur !)
# erreur attrapée dehors
```

Bonus : réécris la même chose avec `@contextmanager` et un générateur (``yield``). Compare les deux versions.

✓ **Critères de réussite**

- `__enter__` renvoie bien `self` (sinon `c` vaudrait `None`).
- L'ordre d'affichage est : connexion → corps → fermeture.
- Dans le test avec erreur, `"fermeture de ..."` apparaît **avant** que l'erreur soit attrapée dehors.

Si tu bloques — indices progressifs

Étape	Indice
<code>__enter__</code>	Affiche le message, puis <code>return self</code> — c'est ce que <code>as c</code> recevra.
<code>__exit__</code>	Il prend 4 paramètres : <code>self, exc_type, exc_val, exc_tb</code> . Tu n'as qu'à afficher le message de fermeture ; les 3 derniers, tu les ignores pour l'instant.
Tester l'erreur	Mets le <code>with</code> dans un <code>try/except ValueError</code> pour que l'erreur ne stoppe pas ton script — et observe l'ordre des affichages.
Bonus <code>@contextmanager</code>	Avant le <code>yield</code> = ce que faisait <code>__enter__</code> ; après le <code>yield</code> = ce que faisait <code>__exit__</code> .

Où ranger : `mini-projets/13-context-managers/connexion.py` .

Lien direct avec la suite : au MP14 (première vraie socket), tu pourras envelopper ta socket dans un `with` pour être sûr qu'elle se ferme toujours. C'est exactement ce context manager, mais avec une vraie connexion réseau dedans.

Quand c'est prêt : `git add .` → `commit` → `push` , puis « c'est poussé ». Ce sera la **fin du Module C** !